

Strategic Optimization of Airbnb Pricing and Property Selection

Chenrui Xu, Zhiming Fan, Zikai Liu, Yunhao Liang, Chenxin Wang, Chirui Huang

December 17, 2024

Abstract

This project addresses the strategic optimization of Airbnb listings by integrating demand modeling and mathematical optimization techniques. We develop an approach that blends Multinomial Logit (MNL) choice modeling with Mixed-Integer Linear Programming (MILP) to jointly determine which listings to feature and how to price them for maximum platform-wide revenue. Using a subset of the New York Airbnb dataset and approximated MNL parameters, our framework captures the interplay between property attributes, price elasticity, and consumer substitution effects. Empirical comparisons reveal that our MNL-based optimization outperforms simpler methods (e.g., selecting only the highest-priced listings) by balancing price points and assortment to enhance overall revenue. The results underscore the potential of data-driven, choice-based optimization in managing short-term rental marketplaces and lay the groundwork for more sophisticated and scalable implementations, including dynamic pricing, improved demand estimation, and heuristic solution methods.

Introduction

As the holiday season approaches, travelers are actively seeking accommodations, with short-term rental platforms like Airbnb offering increasingly popular alternatives to traditional hotels. These platforms have reshaped the lodging industry by providing diverse properties that cater to guests with varying preferences and price sensitivities. Traditional approaches—such as simply offering the most expensive listings—fail to consider how changes in assortment and pricing influence consumer choice. Recognizing this complexity, we propose an optimization framework that integrates choice modeling and mixed-integer programming to guide strategic property selection and pricing decisions. By accounting for factors like property features, consumer preferences, and substitution effects, our method aims to maximize overall platform revenue. Through a demonstration on a sample of Airbnb listings, we highlight how this framework can enhance outcomes compared to more naive strategies, ultimately paving the way for more data-driven, adaptive decision-making in two-sided marketplaces.

Literature Review

Previous research on pricing in two-sided marketplaces often focuses on demand prediction using machine learning methods, including linear regression and random forests [Rezazadeh Kalehbasti et al., 2021]. However, accurate demand forecasts alone do not fully address the strategic challenge of how to select and price properties to maximize platform-wide revenue. Because listings do not operate in isolation, adding or removing a particular property at a given price alters the relative attractiveness of the remaining options. Discrete choice models, particularly MNL, have proven effective in capturing these substitution effects. By expressing choice probabilities as a ratio of exponentiated utilities, MNL models can be integrated into MILP frameworks. Commercial solvers like Gurobi can then efficiently find optimal solutions for large-scale problems that incorporate these choice dynamics.

Data and Experimental Setup

We consider a subset of the New York Airbnb dataset with approximately 37,000 listings. Each entry includes:

- Listing details: *id*, *name*, *host_id*, *host_name*
- Location data: *neighbourhood_group*, *neighbourhood*, *latitude*, *longitude*
- Room information: *room_type*, *price*, *minimum_nights*
- Review metrics: *number_of_reviews*, *last_review*, *reviews_per_month*
- Additional info: *calculated_host_listings_count*, *availability_365*, *license*

We focus on selecting $D = 100$ properties and optimally pricing them. Our methods are:

1. **MNL Choice-based Optimization:** Incorporates consumer utility, price sensitivity, and substitution effects.
2. **Highest Price Selection:** A simpler benchmark that selects the top 100 properties by listed price, ignoring substitution or demand sensitivity.

For modeling purposes, we assume a total demand of 100 units. Properties differ by license validity, number of reviews, and price levels. We use reviews as a proxy for demand, acknowledging its limitations due to the lack of direct booking choice data.

Methodology

Utility Parameter Estimation

In the absence of detailed choice data, we approximate MNL utility parameters using a regression-based approach. We fit a regression model on listing-level features (e.g., host response rate, review scores, price) to estimate relative importance. Although these regression coefficients are not true MNL parameters, we rescale them into indicative utility parameters. An example set of raw regression coefficients and normalized parameters is shown in Table 1.

	Raw Coefficient (Regression)	Normalized Parameter
host_response_rate	3.764	0.37645
is_superhost	5.918	0.59176
identity_verified	7.682	0.76818
review_score	21.931	2.19314
price	-4.748	-0.00475

Table 1: Raw Coefficients and Normalized Utility Parameters (Heuristic)

MNL Choice Modeling

Each property i has a utility function:

$$U_i = \beta_0 + \beta_{\text{license}} \cdot \text{License}_i + \beta_{\text{reviews}} \cdot \text{Reviews}_i + \beta_{\text{price}} \cdot P_i,$$

where P_i is the price chosen for that property, and parameters $(\beta_{\text{license}}, \beta_{\text{reviews}}, \beta_{\text{price}})$ are derived from the regression-based approach.

For a given set of properties, the probability of choosing property i is:

$$\phi_i = \frac{\exp(U_i)}{\sum_{k=1}^N \exp(U_k)}.$$

We discretize the price range for each property into a set of feasible price points $\{P_{i,j}\}$. Thus:

$$U_{i,j} = U_i + \beta_{\text{price}} \cdot P_{i,j}.$$

To keep the optimization linear, we approximate the denominator of the MNL probability by normalizing within each category (small, medium, large) and treating it as constant. Although this reduces modeling fidelity, it allows us to integrate the MNL probabilities into a Mixed-Integer Linear Programming (MILP) framework.

Optimization Formulation

Let $y_{i,j} \in \{0,1\}$ indicate whether property i is selected at price $P_{i,j}$. We want to maximize expected revenue:

$$\max \sum_{i=1}^N \sum_j (P_{i,j} \cdot \phi_{i,j} \cdot D) y_{i,j}.$$

Subject to:

$$\sum_j y_{i,j} \leq 1 \quad \forall i$$

$$\sum_i \sum_j y_{i,j} \leq D$$

Here, D is total demand (e.g., 100 bookings). The probability $\phi_{i,j}$ depends on $U_{i,j}$.

We solve this MILP using a commercial solver (e.g., Gurobi), and leverage insights from recent research on submodular order functions and assortment optimization [Udwani, 2023]. The MNL-based formulation captures substitution effects: if we change the price or set of available properties, the relative attractiveness of each listing changes, thus altering $\phi_{i,j}$.

Experiments and Results

Comparative Approaches

We compare the MNL-based method to the Highest Price Selection strategy:

- **MNL Choice-based Optimization:** Solves a MILP with MNL probabilities.
- **Highest Price Selection:** Sort listings by price and select the top 100 without considering utility or demand shifts.

Sample Results

On a small sample of 10 properties (for demonstration), and then scaling to 100 properties:

- **MNL Choice-based Approach: Average Optimal Price:** \$236.04
- **Highest Price Selection Approach: Average Price:** \$3959.50

While the Highest Price approach yields a higher nominal average price, it does not necessarily maximize revenue once demand sensitivities are factored in. The MNL approach, by incorporating substitution effects and elasticity, can achieve a better overall revenue outcome.

Full-Scale Results

For a demand scenario of 100 bookings, the model’s optimized average prices were approximately:

- Small properties: \$138.00
- Medium properties: \$208.00
- Large properties: \$417.60

This pricing mix yielded a total revenue of around \$21,492. Incorporating MNL-based substitution effects allowed the optimization to balance price points and property selection, potentially outperforming simple heuristics that ignore utility and elasticity.

Discussion

Our approach demonstrates the potential of MNL-based optimization in a complex marketplace like Airbnb. However, limitations exist. We do not have true dynamic demand data, and we rely on reviews as a proxy for demand, which may introduce bias. The linear approximation for the MNL probabilities and the absence of an outside option also limit realism. Future improvements could include better demand estimation, more sophisticated utility parameter calibration (e.g., via Maximum Likelihood Estimation), and modeling of seasonality or time-dependent demand.

Future Directions

Building on the current framework, several enhancements can be explored:

- **Heuristic Algorithms:** Implementing Genetic Algorithms (GA) or Simulated Annealing (SA) can efficiently handle larger datasets and complex solution spaces.
- **Dynamic Demand Modeling:** Incorporating time-series forecasting methods (e.g., ARIMA, LSTM) can capture seasonality and temporal variations, improving dynamic pricing and selection strategies.
- **Comparative Analysis:** Benchmarking the MILP solution against heuristic methods or other optimization heuristics can provide insights into solution quality and computational efficiency.
- **Scalability Testing:** Applying the framework to large datasets (e.g., 5,000 or 37,000 listings) to assess scalability and performance.

Conclusion

We presented a strategic optimization framework for pricing and selecting Airbnb listings. By integrating MNL-based demand modeling into an MILP framework, we captured substitution effects and improved upon naive selection methods. Although the current approach uses approximations and heuristic parameter estimates, it provides a foundation for more robust, data-driven optimization in online marketplaces.

References

Pouya Rezazadeh Kalehbasti, Liubov Nikolenko, and Hoormazd Rezaei. *Airbnb Price Prediction Using Machine Learning and Sentiment Analysis*, page 173–184. Springer International Publishing, 2021. ISBN 9783030840600. doi: 10.1007/978-3-030-84060-0_11. URL http://dx.doi.org/10.1007/978-3-030-84060-0_11.

Rajan Udawani. Submodular order functions and assortment optimization. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 34584–34614. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/udwani23a.html>.

Appendix

Python Code for MNL Optimization

Setup and Library Import:

```
1 # Install Gurobi package
2 !pip install gurobipy
3
4 # Import required libraries
5 import pandas as pd
6 import numpy as np
7 import gurobipy as gp
8 from gurobipy import GRB
9
10 # Gurobi license setup
11 options = {
12      
13      
14      
15 }
```

Listing 1: Library Import and Setup

Data Loading and Preprocessing:

```
1 # Load data
2 data = pd.read_csv("listings.csv")
3
4 # Display first 10 rows
5 pd.set_option('display.max_columns', None)
6 data.head(10)
```

Listing 2: Load and Preview Data

Class Implementation for MNL Optimization:

```
1 class AirbnbPricingOptimizer:
2     """
3     Implements MNL pricing optimization for Airbnb listings with manual
4     parameter input
5     """
6     def __init__(self, utility_params: dict, base_wtb_per_person=50,
7                 price_steps=10, min_price_ratio=0.2, total_demand=100)
8         :
```

```

7      self.utility_params = utility_params
8      self.base_wtb = base_wtb_per_person
9      self.price_steps = price_steps
10     self.min_price_ratio = min_price_ratio
11     self.total_demand = total_demand
12
13     def prepare_optimization_data(self, df: pd.DataFrame):
14         """Prepare data for optimization"""
15         self.df = df.copy()
16         self.n_properties = len(df)
17
18         # Process features
19         self.features = pd.DataFrame()
20         self.features['license'] = df['license'].apply(lambda x: 1 if
21             pd.notna(x) else 0)
22         self.features['number_of_reviews'] = df['number_of_reviews'].
23             fillna(0)
24
25         # Base utilities
26         self.base_utilities = np.zeros(self.n_properties)
27         for feature in ['license', 'number_of_reviews']:
28             self.base_utilities += self.utility_params[feature] * self.
29                 features[feature].values
30
31         # Price points
32         self.baseprice = df['price'].fillna(1).astype(int)
33         self.price_points = [
34             np.linspace(bp * (1 - self.min_price_ratio), bp * (1 + self
35                 .min_price_ratio), self.price_steps)
36             for bp in self.baseprice
37         ]

```

Listing 3: AirbnbPricingOptimizer Class

Optimization Method:

```

1     def optimize_mnl(self):
2         """Run MNL choice-based pricing optimization"""
3         try:
4             with gp.Env(params=options) as env, gp.Model(env=env) as
5                 model:
6                 x = {}
7                 y = {}
8                 for i in range(self.n_properties):
9                     x[i] = model.addVar(vtype=GRB.BINARY, name=f'x_{i}',
10                         )
11                     for j in range(self.price_steps):
12                         y[i, j] = model.addVar(vtype=GRB.BINARY, name=f
13                             'y_{i}_{j}')
14
15                 # Constraints
16                 model.addConstrs(gp.quicksum(y[i, j] for j in range(
17                     self.price_steps)) == x[i] for i in range(self.
18                     n_properties))
19                 model.addConstr(gp.quicksum(x[i] for i in range(self.
20                     n_properties)) <= self.total_demand, name='
21                     total_demand')
22
23                 # Objective Function

```

```

17         exp_utilities = np.exp(self.utilities)
18         denominator = exp_utilities.sum()
19         obj = gp.quicksum(y[i, j] * self.price_points[i][j] *
20                           exp_utilities[i, j] / denominator
21                           for i in range(self.n_properties) for
22                           j in range(self.price_steps))
23
24         model.setObjective(obj, GRB.MAXIMIZE)
25         model.optimize()
26
27     except gp.GurobiError as e:
28         print(f"Optimization error: {e}")

```

Listing 4: Optimization Method

Running the Analysis:

```

1 def run_pricing_analysis(df: pd.DataFrame, utility_params: dict,
2   total_demand=100):
3     """
4     Run both optimization approaches and return results
5     """
6     optimizer = AirbnbPricingOptimizer(
7         utility_params=utility_params,
8         total_demand=total_demand
9     )
10    optimizer.prepare_optimization_data(df)
11    mnl_results = optimizer.optimize_mnl()
12    return mnl_results
13
14 # Define utility parameters
15 utility_params = {
16     'license': 0.3,          # Positive impact for having a license
17     'number_of_reviews': 0.01, # Small positive impact per review
18     'price_sensitivity': -0.001 # Negative impact of price increases
19 }
20
21 # Run analysis
22 mnl_results = run_pricing_analysis(df=data, utility_params=
    utility_params, total_demand=100)

```

Listing 5: Run Pricing Analysis